SEDS 537 Take-Home Midterm

Student Name
Student ID

April 9, 2026

1 Introduction

This report documents the methods, experiments, and results for the five questions in the SEDS 537 take-home midterm.

2 Reproducibility

All experiments are implemented as Python scripts and are intended to be executed from the project root. The project environment uses Python 3.11 with dependencies installed from `requirements.txt`. The environment can be recreated with the following commands:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The shared random seed is defined as `RANDOM_STATE = 42` in `src/common/config.py`. For Question 1, the same 80/20 train-test split is used across all regression experiments. The held-out test set is used only for final evaluation, while model selection for Ridge Regression is performed with 5-fold cross-validation on the training data.

Preprocessing steps are implemented inside scikit-learn pipelines where applicable. This ensures that transformations such as standard scaling and polynomial feature expansion are fit only on the training data and then applied to the test data, avoiding information leakage. Generated figures and metric tables are saved under `results/figures/` and `results/tables/` so that the report outputs can be reproduced by rerunning the corresponding scripts.

For Question 1, the full workflow can be reproduced with:

```
python -m src.q1_regression.run
```

3 Results

3.1 Question 1: Regression - Predicting House Prices

The California Housing dataset was used for the regression task. The target variable is `MedHouseVal`. Exploratory data analysis was performed before model training to inspect feature distributions, missing values, correlations, and relationships between the input features and the target variable.

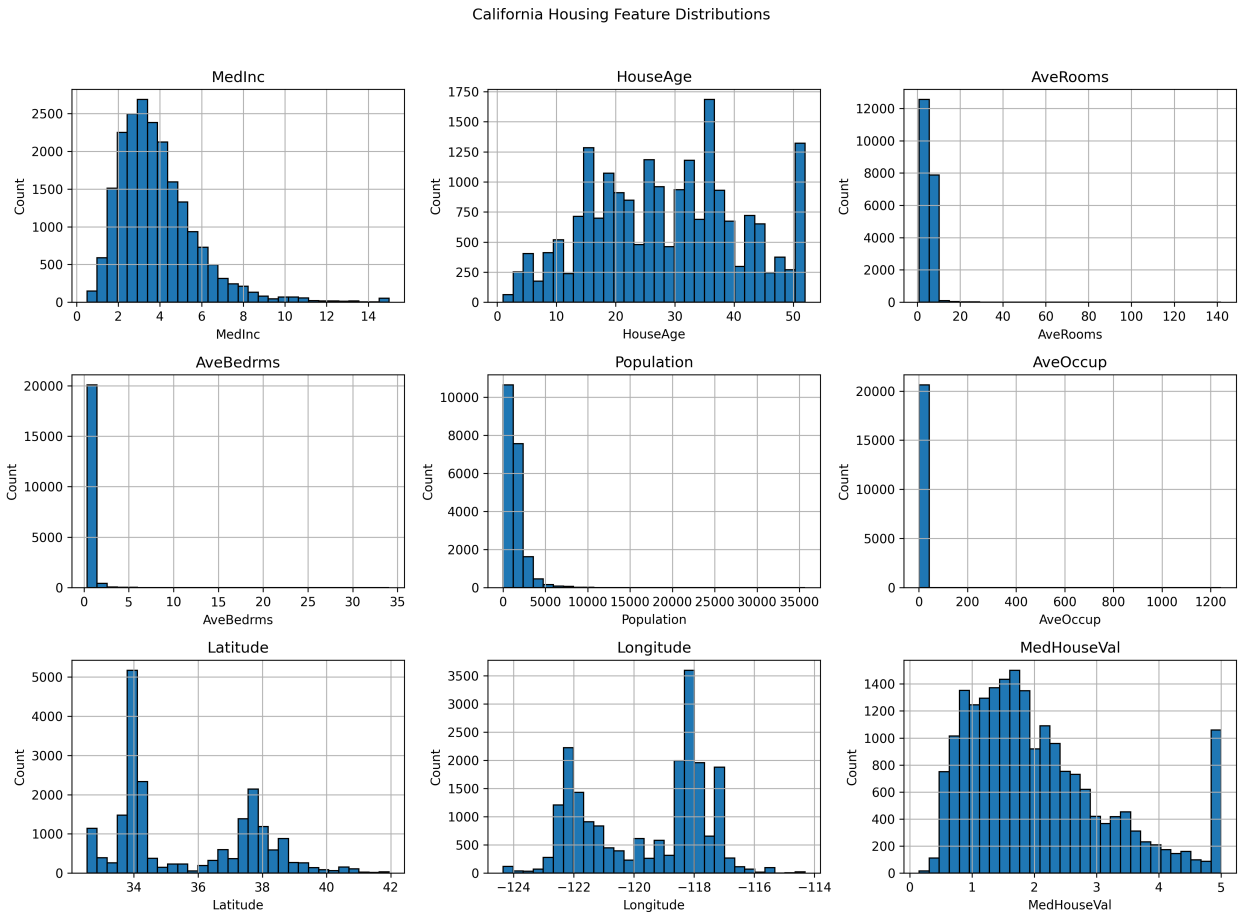

Figure 1: Feature distributions for the California Housing dataset.

The EDA outputs show the distribution of each feature, the linear correlations between variables, and the feature-target relationships that will guide the regression modeling and residual analysis in the next step.

3.1.1 Model Comparison

Table 1 summarizes the regression performance on the held-out test set. Lower RMSE and MAE indicate smaller prediction errors, while higher R^2 indicates better explained variance.

Across the compared models, the original Random Forest Regressor achieved the strongest overall performance, with the scaled Random Forest producing nearly identical results. Polynomial features improved Linear Regression, but they did not outperform the Random Forest model.

3.1.2 Impact of Standard Scaling

Standard scaling had little impact on predictive performance. Linear Regression produced almost identical RMSE, MAE, and R^2 values before and after scaling. This is expected because scaling changes the magnitude of the coefficients but does not substantially change the fitted linear relationship. Random Forest also showed almost no change, which is expected because tree-based models are generally not sensitive to feature scale.

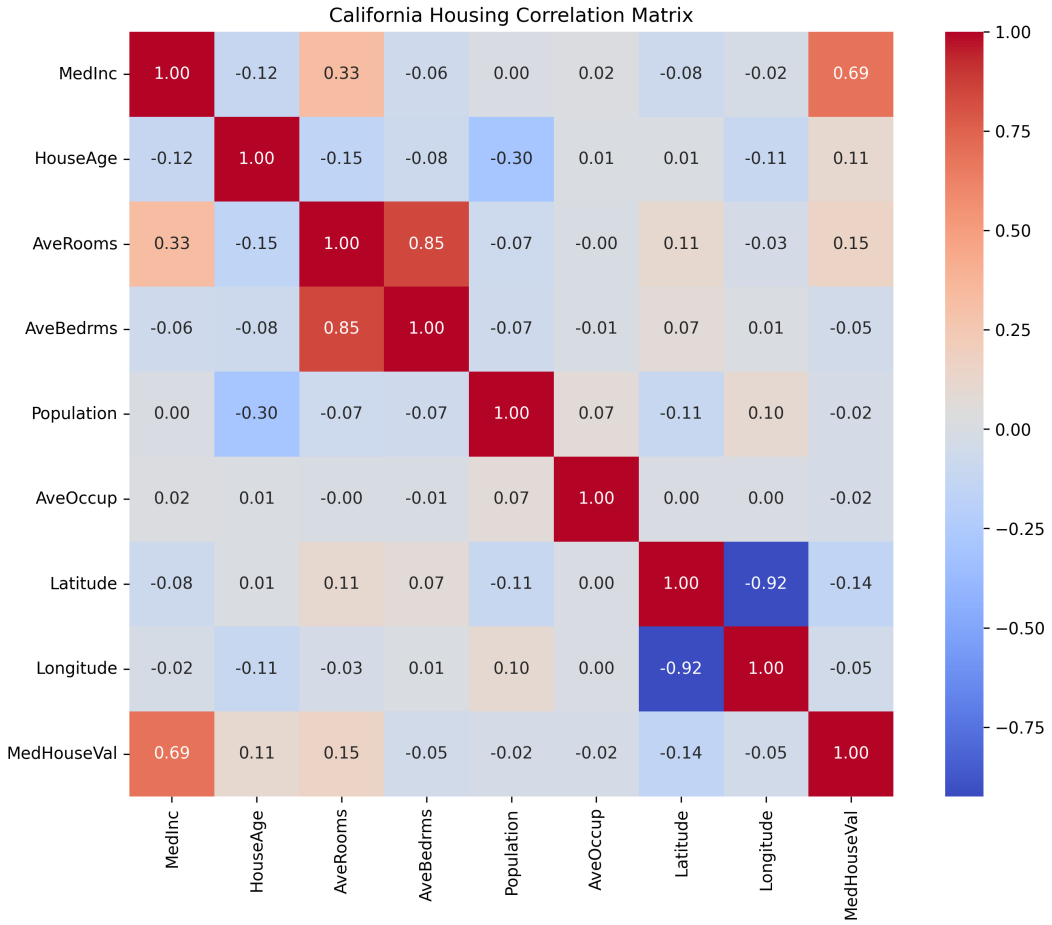

Figure 2: Correlation matrix for the California Housing dataset.

For Ridge Regression, scaling changed the selected alpha value from 10.0 to 0.001, showing that feature scale affected the regularization process. However, the final test performance remained very close to the unscaled Ridge model. Therefore, standard scaling improved the comparability of feature scales and affected the Ridge hyperparameter selection, but it did not lead to a meaningful improvement in predictive performance for this dataset and split.

3.1.3 Impact of Polynomial Features

Degree-2 polynomial features improved the Linear Regression model compared with the original baseline. The polynomial linear model reached an RMSE of 0.6814, MAE of 0.4670, and R^2 of 0.6457, while the original Linear Regression model had an RMSE of 0.7456, MAE of 0.5332, and R^2 of 0.5758. This suggests that the California Housing dataset contains nonlinear relationships and feature interactions that a simple linear model cannot capture.

Polynomial Ridge Regression did not improve performance. Its R^2 was 0.5733 and the selected alpha was 1000.0, indicating that strong regularization was selected after the polynomial expansion. This likely reduced the benefit of the additional polynomial terms.

The Polynomial Random Forest Regressor achieved an R^2 of 0.7991, which was slightly worse than the original Random Forest result of 0.8062. This is reasonable because Random Forest can already model nonlinear relationships and interactions through tree splits, so manually adding

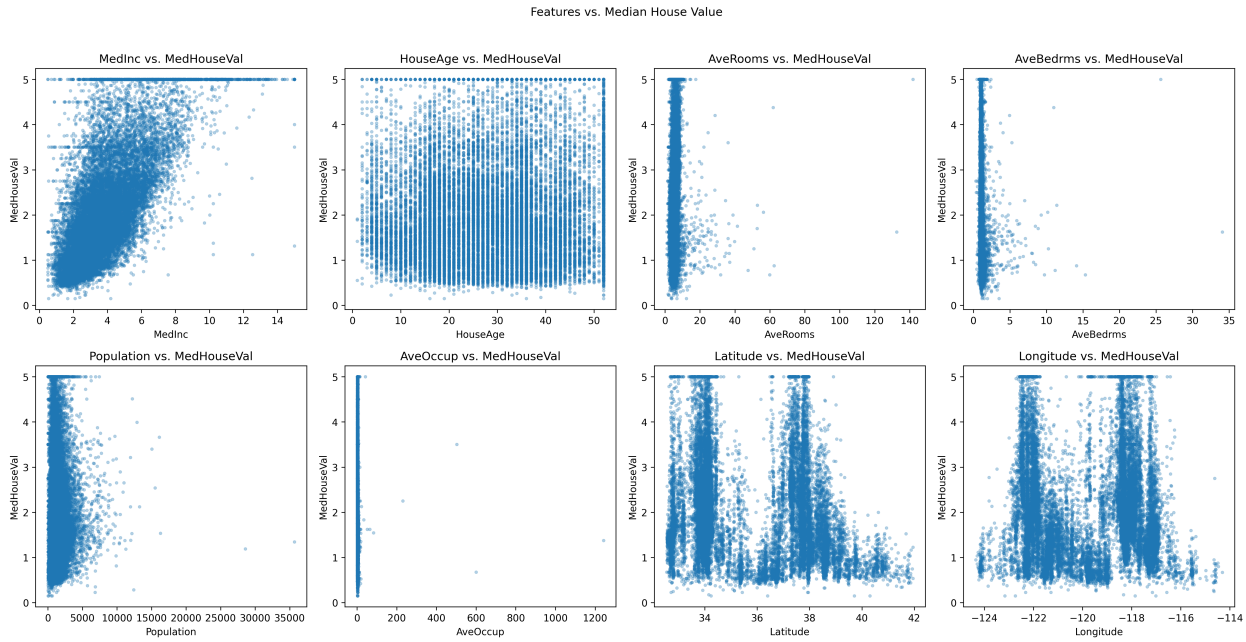

Figure 3: Scatter plots of each feature against median house value.

Table 1: Question 1 regression model comparison on the held-out test set.

Model	Preprocessing	RMSE	MAE	R^2	Selected Alpha
Random Forest Regressor	None	0.5039	0.3266	0.8062	–
Ridge Regression	None	0.7450	0.5332	0.5764	10.0
Linear Regression	None	0.7456	0.5332	0.5758	–
Random Forest Regressor	Standard scaling	0.5038	0.3265	0.8063	–
Ridge Regression	Standard scaling	0.7456	0.5332	0.5758	0.001
Linear Regression	Standard scaling	0.7456	0.5332	0.5758	–
Polynomial Random Forest Regressor	Degree-2 polynomial	0.5131	0.3345	0.7991	–
Polynomial Linear Regression	Degree-2 polynomial	0.6814	0.4670	0.6457	–
Polynomial Ridge Regression	Degree-2 polynomial	0.7478	0.5291	0.5733	1000.0

polynomial features does not necessarily help and can add unnecessary feature complexity. Overall, polynomial features were useful for the linear model, but Random Forest remained the strongest model.

3.1.4 Residual Analysis

The residual plots in Figure 4 compare the error patterns of the main regression models using the same axis limits across subplots. With this shared scale, the vertical spread around the zero-residual line is the main feature to compare. The linear models show visible structure in their residuals, suggesting that a purely linear relationship does not fully capture the patterns in the California Housing data. The polynomial linear model reduces some of this structure, which is consistent with its improvement over the baseline Linear Regression model.

The diagonal upper boundary visible in the residual plots is caused by the target variable `MedHouseVal` being capped at 5.0. Since residuals are calculated as actual value minus fitted value,

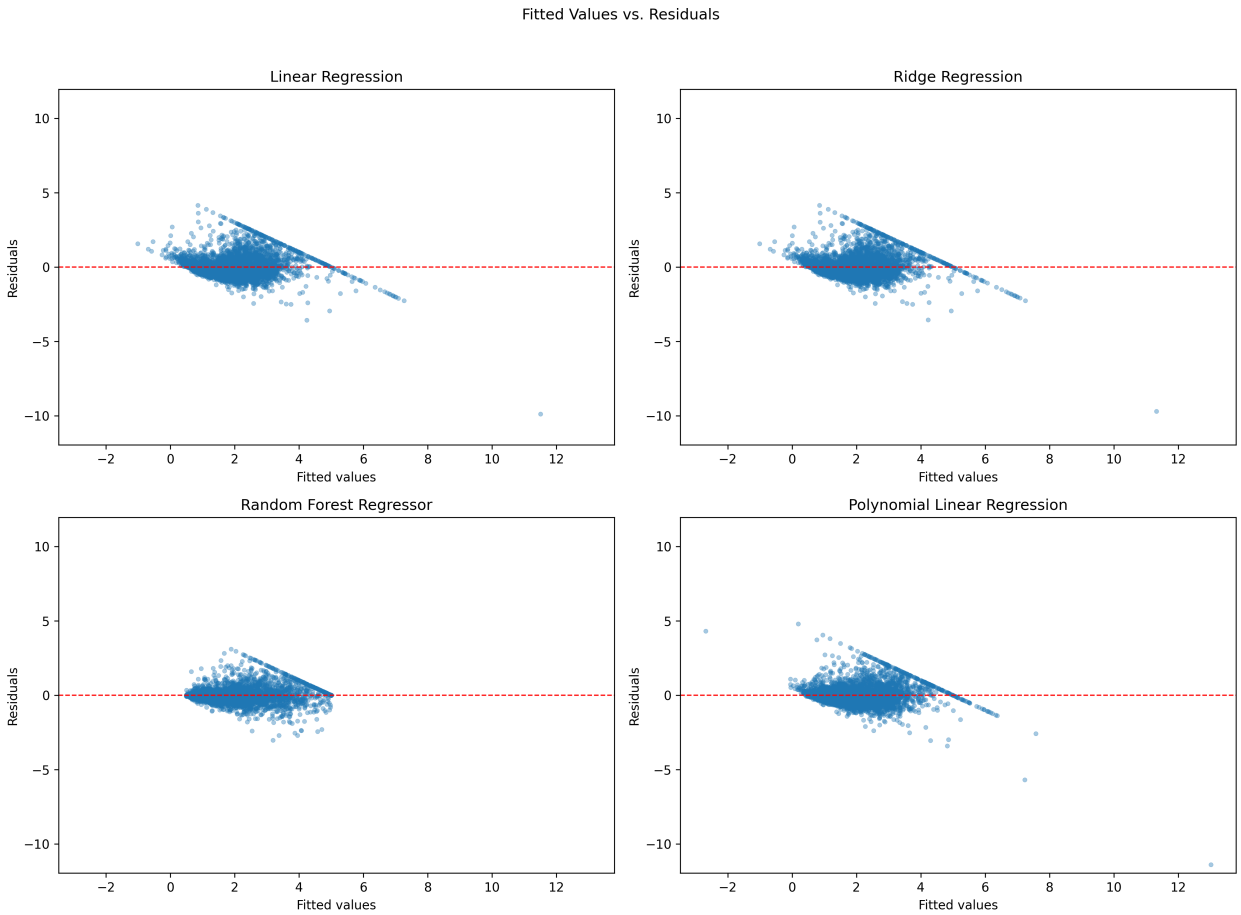

Figure 4: Fitted values versus residuals for representative regression models.

the maximum possible positive residual is limited by $5.0 - \hat{y}$. This creates a downward-sloping boundary and shows that high-value districts are affected by the artificial target cap. Overall, the residual plots should be interpreted together with the metric table: Random Forest still performs best because it has the lowest RMSE and MAE and the highest R^2, even though some residual structure remains at higher fitted values.

3.1.5 Interpretability vs. Predictive Performance

Linear Regression and Ridge Regression are the most interpretable models because their coefficients directly describe the direction and magnitude of feature effects. However, their predictive performance was weaker, with R^2 values around 0.576. Polynomial Linear Regression improved performance by adding nonlinear feature interactions, reaching an R^2 of 0.6457, but this reduced interpretability because the model includes many transformed and interaction terms.

Random Forest achieved the best predictive performance, with the lowest RMSE and MAE and the highest R^2 score of 0.8062. This suggests that nonlinear relationships and feature interactions are important for the California Housing dataset. The trade-off is that Random Forest is less directly interpretable than the linear models because predictions are based on many decision trees rather than a small set of coefficients. Overall, Random Forest is the preferred model for prediction, while Linear Regression and Ridge Regression are more useful for simpler interpretation.

3.2 Question 2: Classification - Handling Class Imbalance

3.2.1 Dataset Description

For Question 2, the Credit Card Fraud Detection dataset was used. The dataset contains 31 columns in total: 28 anonymized PCA-transformed numerical features (`V1` to `V28`), the `Time` feature, the `Amount` feature, and the target variable `Class`. The target is binary, where `Class = 0` represents a legitimate transaction and `Class = 1` represents a fraudulent transaction.

The anonymized variables `V1` to `V28` do not have direct real-world meanings because they were transformed using Principal Component Analysis (PCA) to protect sensitive financial information. However, they still retain useful information for fraud detection. The `Amount` column represents the transaction amount, while `Time` records the number of seconds elapsed since the first transaction in the dataset.

3.2.2 Class Distribution

The dataset is highly imbalanced. There are 284,315 legitimate transactions (99.8273%) and only 492 fraudulent transactions (0.1727%). This confirms that the fraud class is extremely underrepresented compared with the legitimate class.

Such an imbalance makes accuracy alone a misleading evaluation metric, because a classifier could achieve very high accuracy by predicting almost every transaction as legitimate. For this reason, Question 2 focuses on precision, recall, F1-score, ROC-AUC, confusion matrix analysis, and resampling methods such as SMOTE and random undersampling.

3.2.3 Baseline Models

An 80/20 stratified train-test split was used so that the severe class imbalance was preserved in both partitions. Three baseline classifiers were then trained on the original imbalanced training data: Logistic Regression, Random Forest, and XGBoost. Their performance on the held-out test set is summarized in Table 2.

Table 2: Question 2 baseline classifier comparison on the imbalanced test set.

Model	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.8267	0.6327	0.7168	0.9605
Random Forest	0.9419	0.8265	0.8804	0.9623
XGBoost	0.9277	0.7857	0.8508	0.9616

The baseline results show that the tree-based models perform substantially better than Logistic Regression on this dataset. Logistic Regression achieves a strong ROC-AUC, but its recall is only 0.6327, meaning that it misses a relatively large portion of fraudulent transactions. This is a common issue in highly imbalanced problems, where a simple linear model can separate many normal transactions well but still fail to recover enough minority-class examples.

Random Forest produced the strongest baseline result overall, with the highest precision, recall, F1-score, and ROC-AUC among the three models. Its confusion matrix shows only 17 false negatives and 5 false positives on the test set, which indicates a better balance between fraud detection and avoiding unnecessary fraud alerts. XGBoost also performed strongly and clearly outperformed Logistic Regression, but it remained slightly behind Random Forest in recall and F1-score for this baseline configuration.

At this stage, the main finding is that model choice already matters even before applying resampling. However, because the dataset remains extremely imbalanced, the next step is to test whether SMOTE or random undersampling can further improve minority-class detection, especially recall, without creating too many false positives.

3.2.4 Resampling Results

SMOTE and random undersampling were both applied only to the training set after the stratified split, while the test set remained unchanged. SMOTE produced a balanced training set with 227,451 legitimate transactions and 227,451 fraudulent transactions. Random undersampling reduced the majority class down to the minority class size, producing a much smaller balanced training set with 394 legitimate transactions and 394 fraudulent transactions. Table 3 compares the baseline, SMOTE, and random undersampling results on the held-out test set.

Table 3: Question 2 baseline, SMOTE, and random undersampling comparison on the held-out test set.

Model	Training Data	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	Original imbalanced data	0.8267	0.6327	0.7168	0.9605
Logistic Regression	SMOTE-balanced training data	0.1341	0.8980	0.2334	0.9765
Logistic Regression	Randomly undersampled training data	0.0516	0.9184	0.0977	0.9752
Random Forest	Original imbalanced data	0.9419	0.8265	0.8804	0.9623
Random Forest	SMOTE-balanced training data	0.8265	0.8265	0.8265	0.9685
Random Forest	Randomly undersampled training data	0.0423	0.9184	0.0809	0.9780
XGBoost	Original imbalanced data	0.9277	0.7857	0.8508	0.9616
XGBoost	SMOTE-balanced training data	0.7456	0.8673	0.8019	0.9820
XGBoost	Randomly undersampled training data	0.0348	0.9184	0.0671	0.9741

The resampling experiments show that both methods strongly shift the precision-recall trade-off toward higher recall. For Logistic Regression, SMOTE increased recall from 0.6327 to 0.8980, and random undersampling increased it further to 0.9184. However, precision dropped sharply to 0.1341 with SMOTE and to 0.0516 with undersampling. This caused the F1-score to fall well below the baseline value. In other words, the model found more fraud cases, but it also produced many more false alarms.

For Random Forest, SMOTE did not improve the overall balance of performance. Recall remained at 0.8265, while precision dropped from 0.9419 to 0.8265, lowering the F1-score from 0.8804 to 0.8265. Random undersampling pushed recall to 0.9184, but precision collapsed to 0.0423 because the model flagged far too many legitimate transactions as fraud. As a result, the undersampled Random Forest performed much worse than both the baseline and SMOTE versions under the F1-score criterion.

For XGBoost, SMOTE increased recall from 0.7857 to 0.8673 and improved ROC-AUC from 0.9616 to 0.9820. However, precision decreased from 0.9277 to 0.7456, and the F1-score fell from 0.8508 to 0.8019. Random undersampling again forced recall up to 0.9184, but precision dropped to 0.0348 and the F1-score fell to 0.0671. This indicates that the undersampled XGBoost model became far too aggressive in predicting fraud.

Overall, the current results indicate that SMOTE is useful when the goal is to recover more fraud cases, especially for Logistic Regression and XGBoost, but it does not automatically improve the most balanced model. Random undersampling increased recall for all three classifiers, yet it damaged precision so severely that all three undersampled models had very poor F1-scores. Among

all experiments completed so far, the baseline Random Forest still provides the strongest overall trade-off between fraud detection and false alarms.

3.2.5 ROC Curve and Error Analysis

Figure 5 shows the ROC curve for the baseline Random Forest model, which produced the strongest overall classification result among the baseline, SMOTE, and random undersampling experiments.

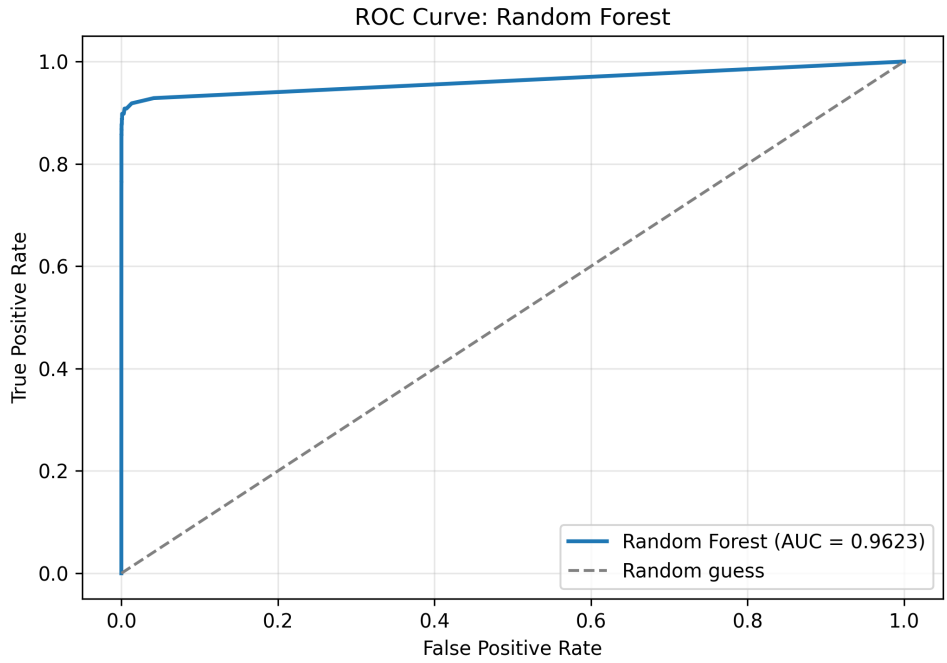

Figure 5: ROC curve for the baseline Random Forest classifier on the Question 2 test set.

The ROC curve plots the true positive rate against the false positive rate across different decision thresholds. A curve closer to the top-left corner indicates better class separation, while the diagonal reference line represents random guessing. The baseline Random Forest achieved an ROC-AUC of 0.9623, which indicates strong discrimination between legitimate and fraudulent transactions.

This result is consistent with the confusion matrix and F1-score analysis. The baseline Random Forest combines high precision (0.9419) with strong recall (0.8265), so it catches most fraud cases while keeping false alarms low. Compared with the resampled models, its ROC-AUC is not always the highest, but its threshold-based performance at the default decision rule is more balanced. This is why it remains the strongest overall model for the current fraud detection setup.

3.2.6 Discussion

In the credit card fraud detection setting, there is a clear trade-off between precision and recall. A model with higher recall can identify more fraudulent transactions, but this often comes at the cost of lower precision, which means more legitimate transactions are incorrectly flagged as fraud. In practice, this trade-off matters because the two types of classification error have different business consequences.

For this dataset, a false negative is generally the more costly error. A false negative means that a fraudulent transaction is missed and allowed to pass as legitimate, which can lead to direct

financial loss, customer harm, and delayed fraud response. A false positive is also undesirable because it may block a legitimate transaction, inconvenience the customer, and create unnecessary review work, but this cost is usually smaller than failing to stop an actual fraudulent charge.

The resampling experiments illustrate this trade-off clearly. SMOTE and random undersampling both pushed recall upward, but they did so by sharply increasing false positives and reducing precision. This behavior may be acceptable in some fraud monitoring systems where catching as many fraud cases as possible is the top priority. However, in the current experiment, the loss of precision was too severe, especially for the undersampled models, which led to much weaker F1-scores.

Overall, the baseline Random Forest is the most appropriate model among the tested configurations. It achieves the best balance between fraud detection and false alarms, with high precision, strong recall, the highest F1-score, and a strong ROC-AUC. Therefore, it is the preferred model for this dataset under a balanced evaluation criterion, while a higher-recall resampled model could still be considered in a setting where minimizing false negatives is the primary objective.

3.3 Question 3: Dimensionality Reduction and Visualisation

3.3.1 Dataset Description

For Question 3, the MNIST handwritten digit dataset is used. Each sample is a grayscale image of a digit from 0 to 9 with resolution 28×28, which corresponds to 784 pixel features after flattening the image into a vector. This makes MNIST a high-dimensional dataset that is suitable for studying dimensionality reduction and latent-space structure.

3.3.2 PCA Analysis

This section will report PCA results for 2-dimensional and 50-dimensional representations, including the explained variance ratio for the first 10 principal components.

3.3.3 t-SNE Analysis

This section will present 2-dimensional t-SNE embeddings using perplexity values 30 and 50 and discuss how well the digit classes separate visually.

3.3.4 Embedding Visualisations

This section will include 2D scatter plots for PCA and t-SNE embeddings, with points coloured by digit label.

3.3.5 k-NN Classification Comparison

This section will compare 5-fold cross-validation accuracy for k-NN ($k = 5$) on the original 784-dimensional features, PCA-reduced 50-dimensional features, and t-SNE 2-dimensional features.

3.3.6 Discussion

This section will compare the strengths and weaknesses of PCA and t-SNE for visualisation quality, dimensionality reduction, and downstream classification performance.